

Geeks Training DP Problem

Geek's Training — Dynamic Programming

This is a classic **DP (Dynamic Programming)** problem.

The main idea is:

On every day, choose the activity giving the maximum points, **but you cannot choose the same activity as yesterday.**

1. What is the problem asking?

We have n days and **3 activities**:

Activity	Index
Running	0
Fighting	1
Learning	2

For every day, we know how many points each activity gives.

For example:

```
Day 0: [1, 2, 5]
      R  F  L
Day 1: [3, 1, 1]
Day 2: [3, 3, 3]
```

We have to select **exactly one activity per day**.

Rule

We cannot perform the same activity on consecutive days.

For example:

```
Day 0 → Learning
Day 1 → Learning ❌
```

But:

```
Day 0 → Learning
Day 1 → Running  ✅
Day 2 → Fighting  ✅
```

We need the **maximum possible total points**.

2. Key Observation

The important thing is that the choice we make today depends only on:

1. **Which day we are on**
2. **What activity we did yesterday**

We don't need to remember the entire history.

For example, suppose:

```
Today = Day 5
Yesterday = Running
```

Then today we only care about:

```
Fighting
Learning
```

We don't care what we did on Day 0, Day 1, Day 2, etc.

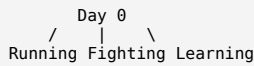
This is exactly the kind of situation where **Dynamic Programming** works.

3. Brute Force / Recursion

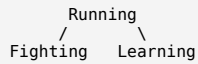
Let's first think about the simplest approach.

On every day, we have up to 3 choices.

Suppose on Day 0:

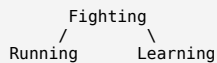


Then on Day 1, if we selected Running on Day 0:



We cannot choose Running again.

Similarly, if Day 0 was Fighting:



So the number of possibilities grows exponentially.

Recursive idea

We can define:

```
solve(day, lastActivity)
```

Meaning:

What is the maximum score we can obtain from **day** onward if **lastActivity** was performed yesterday?

For every day, try all activities except **lastActivity**.

Recurrence

```

solve(day, last)
=
max(
    points[day][activity] + solve(day+1, activity)
)
  
```

where:

```
activity != last
```

4. Why Brute Force is Too Slow

Suppose we have:

```
n = 5
```

There are roughly:

```
3 × 2 × 2 × 2 × 2
```

possibilities.

For large n:

```
n = 100000
```

this becomes enormous.

So recursion alone will give **TLE**.

But notice something interesting.

The same state can be calculated multiple times.

For example:

```
solve(5, Running)
```

might be reached through multiple different paths.
 Instead of calculating it again, we can **store its answer**.
 That's memoization.

5. Memoization

We create:

```
C++
dp[day][lastActivity]
```

where:

```
dp[day][last]
```

means:

Maximum points obtainable from **day** to the end when **last** was the activity performed on the previous day.

There are only:

```
n × 3
```

states.

Each state tries only 2 activities.

Therefore:

```
Time = 0(n × 3 × 2)
      = 0(n)

Space = 0(n × 3)
      = 0(n)
```

Memoization is already efficient enough for $n = 10^5$.

But we can do even better in terms of space.

6. Tabulation — Bottom-Up DP

Instead of recursion, we can build the answer day by day.

Define:

```
dp[i][j]
```

as:

Maximum points we can get up to day **i**, if activity **j** is performed on day **i**.

Here:

```
j = 0 → Running
j = 1 → Fighting
j = 2 → Learning
```

Day 0

There is no previous day.

So we can directly take the points:

```
dp[0][0] = mat[0][0]
dp[0][1] = mat[0][1]
dp[0][2] = mat[0][2]
```

For:

```
[1, 2, 5]
```

we get:

```
dp[0] = [1, 2, 5]
```

7. Transition

Now consider Day i .

Suppose we want to do **Running** today.

We cannot have done Running yesterday.

Therefore yesterday must have been:

Fighting OR Learning

So:

```
dp[i][0]
=
mat[i][0] + max(dp[i-1][1], dp[i-1][2])
```

Similarly, if today we do Fighting:

```
dp[i][1]
=
mat[i][1] + max(dp[i-1][0], dp[i-1][2])
```

And if today we do Learning:

```
dp[i][2]
=
mat[i][2] + max(dp[i-1][0], dp[i-1][1])
```

Therefore:

```
dp[i][0] = mat[i][0] + max(dp[i-1][1], dp[i-1][2])
dp[i][1] = mat[i][1] + max(dp[i-1][0], dp[i-1][2])
dp[i][2] = mat[i][2] + max(dp[i-1][0], dp[i-1][1])
```

Finally:

```
answer = max(dp[n-1][0],
             dp[n-1][1],
             dp[n-1][2])
```

8. Why This DP Works

Imagine:

```
Yesterday:
Running = 10
Fighting = 20
Learning = 15
```

Today we want Running.

We cannot use yesterday's Running value:

```
Today Running
  ↓
Yesterday Running ❌
```

So we choose the best valid previous state:

```
max(Fighting, Learning)
= max(20, 15)
= 20
```

Then add today's Running points.

That's the entire DP transition.

9. C++ Code — Tabulation

```
C++
class Solution {
public:
    int maximumPoints(vector<vector<int>>& mat, int n) {
        vector<vector<int>> dp(n, vector<int>(3, 0));

        // Base case: Day 0
        dp[0][0] = mat[0][0]; // Running
```

```

dp[0][1] = mat[0][1]; // Fighting
dp[0][2] = mat[0][2]; // Learning

// Fill DP table
for (int i = 1; i < n; i++) {

    // Today = Running
    dp[i][0] = mat[i][0] +
               max(dp[i - 1][1], dp[i - 1][2]);

    // Today = Fighting
    dp[i][1] = mat[i][1] +
               max(dp[i - 1][0], dp[i - 1][2]);

    // Today = Learning
    dp[i][2] = mat[i][2] +
               max(dp[i - 1][0], dp[i - 1][1]);

}

// Answer = best activity on the last day
return max({
    dp[n - 1][0],
    dp[n - 1][1],
    dp[n - 1][2]
});
}
};

```

10. Code Explained Step-by-Step

Step 1: Create DP table

```

C++
vector<vector<int>>> dp(n, vector<int>(3, 0));

```

It looks like:

	Running	Fighting	Learning
Day 0	0	0	0
Day 1	0	0	0
Day 2	0	0	0
...			

Step 2: Initialize Day 0

```

C++
dp[0][0] = mat[0][0];
dp[0][1] = mat[0][1];
dp[0][2] = mat[0][2];

```

Because there is no previous day, **anything is allowed**.

Step 3: Calculate each future day

For Running:

```

C++
dp[i][0] = mat[i][0] +
           max(dp[i - 1][1], dp[i - 1][2]);

```

Translation:

Today's Running points + best score from yesterday where yesterday was NOT Running.

For Fighting:

```

C++
dp[i][1] = mat[i][1] +
           max(dp[i - 1][0], dp[i - 1][2]);

```

For Learning:

```

C++
dp[i][2] = mat[i][2] +
           max(dp[i - 1][0], dp[i - 1][1]);

```

11. Complete Dry Run

Given:

```

mat = [
    [1, 2, 5],
    [3, 1, 1],

```

```
[3, 3, 3]
]
```

Let's label:

	R	F	L
Day 0	[1, 2, 5]		
Day 1	[3, 1, 1]		
Day 2	[3, 3, 3]		

Day 0

No previous day.

So:

```
dp[0][R] = 1
dp[0][F] = 2
dp[0][L] = 5
```

Table:

	R	F	L
Day 0	1	2	5

Day 1

Day 1 values:

```
[3, 1, 1]
```

If we choose Running

Today's Running gives:

```
3
```

Yesterday cannot be Running.

So:

```
max(dp[0][F], dp[0][L])
= max(2, 5)
= 5
```

Therefore:

```
dp[1][R] = 3 + 5
          = 8
```

The path is:

```
Day 0 → Learning = 5
Day 1 → Running  = 3
Total = 8
```

If we choose Fighting

Today's Fighting:

```
1
```

Yesterday cannot be Fighting.

Therefore:

```
max(dp[0][R], dp[0][L])
= max(1, 5)
= 5
```

So:

```
dp[1][F] = 1 + 5
          = 6
```

Path:

```
Day 0 → Learning = 5
Day 1 → Fighting = 1
Total = 6
```

If we choose Learning

Today's Learning:

```
1
```

Yesterday cannot be Learning.

Therefore:

```
max(dp[0][R], dp[0][F])
= max(1, 2)
= 2
```

So:

```
dp[1][L] = 1 + 2
          = 3
```

Path:

```
Day 0 → Fighting = 2
Day 1 → Learning = 1
Total = 3
```

Therefore Day 1 becomes:

	R	F	L
Day 0	1	2	5
Day 1	8	6	3

Day 2

Day 2:

```
[3, 3, 3]
```

Day 2 → Running

Today's Running:

```
3
```

Yesterday cannot be Running.

So:

```
max(dp[1][F], dp[1][L])
= max(6, 3)
= 6
```

Therefore:

```
dp[2][R] = 3 + 6
          = 9
```

Best path:

```
Day 0 → Learning = 5
Day 1 → Fighting = 1
Day 2 → Running = 3
Total = 9
```

Day 2 → Fighting

Today's Fighting:

```
3
```

Yesterday cannot be Fighting.

So:

```
max(dp[1][R], dp[1][L])
= max(8, 3)
= 8
```

Therefore:

```
dp[2][F] = 3 + 8
= 11
```

Best path:

```
Day 0 → Learning = 5
Day 1 → Running = 3
Day 2 → Fighting = 3

Total = 11
```

Day 2 → Learning

Today's Learning:

```
3
```

Yesterday cannot be Learning.

So:

```
max(dp[1][R], dp[1][F])
= max(8, 6)
= 8
```

Therefore:

```
dp[2][L] = 3 + 8
= 11
```

Best path:

```
Day 0 → Learning = 5
Day 1 → Running = 3
Day 2 → Learning = 3 ❌
```

Wait — this appears to be a problem if we interpret the stored path incorrectly.

The $dp[1][R] = 8$ path ends with **Running**, not Learning.

So when we choose Learning on Day 2:

```
Day 0 → Learning
Day 1 → Running
Day 2 → Learning
```

This is perfectly valid because Learning was done on Day 0, **not on the consecutive previous day**.

Thus:

```
5 + 3 + 3 = 11
```

So:

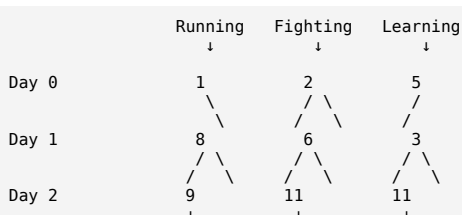
```
dp[2] = [9, 11, 11]
```

Finally:

```
answer = max(9, 11, 11)
= 11
```

12. Complete DP Table

The entire calculation can be visualized as:



Maximum = 11

The winning route can be:

```
Day 0
Learning = 5
  ↓
Day 1
Running = 3
  ↓
Day 2
Fighting = 3
Total = 5 + 3 + 3
      = 11
```

13. Space Optimization

Notice something important.

To calculate Day **i**, we only need **Day i-1**.

We don't actually need the entire DP table.

Instead of:

```
dp[0]
dp[1]
dp[2]
dp[3]
...
dp[n-1]
```

we can maintain only:

```
previous day
current day
```

So space becomes:

```
O(1)
```

Optimized C++ Code

```
C++
class Solution {
public:
    int maximumPoints(vector<vector<int>>& mat, int n) {
        int prevR = mat[0][0];
        int prevF = mat[0][1];
        int prevL = mat[0][2];

        for (int i = 1; i < n; i++) {
            int currR = mat[i][0] + max(prevF, prevL);
            int currF = mat[i][1] + max(prevR, prevL);
            int currL = mat[i][2] + max(prevR, prevF);

            prevR = currR;
            prevF = currF;
            prevL = currL;
        }

        return max({prevR, prevF, prevL});
    }
};
```

This is the version I would recommend submitting.

14. Why curr Variables Are Important

Notice this:

```
C++
int currR = mat[i][0] + max(prevF, prevL);
int currF = mat[i][1] + max(prevR, prevL);
int currL = mat[i][2] + max(prevR, prevF);
```

We calculate all three using the **old previous-day values**.

Only after calculating everything do we update:

```
C++
prevR = currR;
prevF = currF;
prevL = currL;
```

If we directly modified **prevR** before calculating **currF**, we could accidentally use a value from the **current day** instead of yesterday.

15. Complexity

Brute Force

```
Time:  $O(2^n)$  approximately
Space:  $O(n)$  recursion stack
```

Memoization

There are $n \times 3$ states:

```
Time:  $O(n)$ 
Space:  $O(n)$ 
```

Tabulation

```
Time:  $O(n)$ 
Space:  $O(n)$ 
```

Space-Optimized DP

```
Time:  $O(n)$ 
Space:  $O(1)$ 
```

Since:

```
 $n \leq 10^5$ 
```

the **$O(n)$ time + $O(1)$ space** solution is ideal.

16. The DP Pattern to Remember

This problem teaches a very useful DP pattern:

Current choice depends on the previous choice.

Whenever you see:

```
Choose something every day
+
Cannot choose the same thing consecutively
+
Maximize/minimize total score
```

think:

```
dp[day][last choice]
```

For this problem:

```
dp[day] — Running
         — Fighting
         — Learning
```

And the transition is simply:

```
Today's activity
+
best of yesterday's DIFFERENT activities
```

So the core formula to remember is:

```
dp[i][activity]
=
points[i][activity]
```

```
+
max(previous states excluding activity)
```

That's the entire intuition behind **Geek's Training**.

Absolutely. For **Geek's Training**, the memoization approach is the recursive version of the same DP.

1. Memoization State

We define:

```
C++
dp[day][last]
```

Meaning:

Maximum points we can get from **day** to the end, when **last** is the activity we performed on the previous day.

We use:

```
last = 0 → Running
last = 1 → Fighting
last = 2 → Learning
last = 3 → No activity / nothing was done before
```

Using **last = 3** for the initial call is convenient because on Day 0, **all 3 activities are allowed**.

2. Memoization Code

```
C++
class Solution {
public:
    int solve(int day, int last, vector<vector<int>>& mat,
              vector<vector<int>>& dp) {
        // Base case
        if (day == 0) {
            int maxi = 0;
            for (int activity = 0; activity < 3; activity++) {
                if (activity != last) {
                    maxi = max(maxi, mat[0][activity]);
                }
            }
            return maxi;
        }
        // Already calculated
        if (dp[day][last] != -1) {
            return dp[day][last];
        }
        int maxi = 0;
        // Try all 3 activities
        for (int activity = 0; activity < 3; activity++) {
            // Cannot perform same activity as yesterday
            if (activity != last) {
                int points = mat[day][activity]
                    + solve(day - 1, activity, mat, dp);
                maxi = max(maxi, points);
            }
        }
        return dp[day][last] = maxi;
    }
    int maximumPoints(vector<vector<int>>& mat, int n) {
        // last = 0,1,2 are activities
        // last = 3 means no previous activity
        vector<vector<int>> dp(n, vector<int>(4, -1));
        return solve(n - 1, 3, mat, dp);
    }
};
```

3. Understand the Recursive Call

We start with:

```
C++
```

```
solve(n - 1, 3, mat, dp)
```

For the sample:

```
mat =
Day 0 → [1, 2, 5]
Day 1 → [3, 1, 1]
Day 2 → [3, 3, 3]
```

We start:

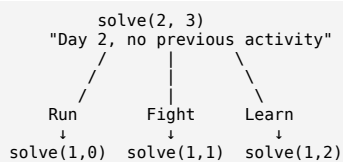
```
solve(2, 3)
```

Meaning:

Find the maximum score from Day 2 to Day 0, and there is no activity restriction initially.

4. Tree Diagram

The recursion starts like this:



Remember:

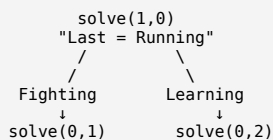
```
0 = Running
1 = Fighting
2 = Learning
3 = Nothing
```

Expand solve(1, 0)

last = 0 means:

Yesterday we did Running.

Therefore today we **cannot choose Running**.

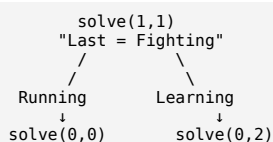


Expand solve(1, 1)

last = 1 means:

Yesterday we did Fighting.

Therefore:

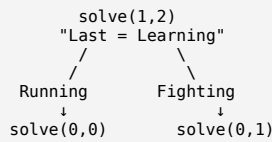


Expand solve(1, 2)

last = 2 means:

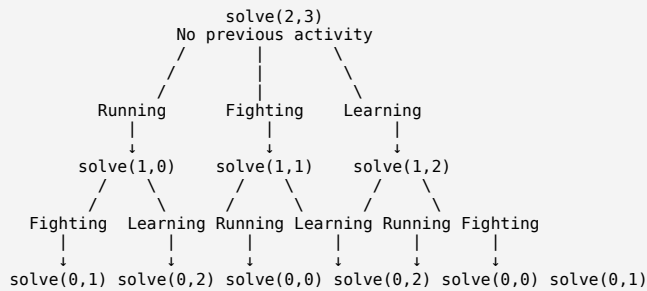
Yesterday we did Learning.

Therefore:



5. Complete Recursion Tree

Putting everything together:



Now we reach Day 0.

6. What Happens at Day 0?

Consider:

```
solve(0,1)
```

last = 1 means Fighting was performed on Day 1.

So Day 0 cannot be Fighting.

Day 0:

```
Running = 1
Fighting = 2
Learning = 5
```

Therefore:

```
solve(0,1)
= max(1, 5)
= 5
```

Similarly:

```
solve(0,2)
```

means Learning is forbidden.

So:

```
max(Running, Fighting)
= max(1, 2)
= 2
```

And:

```
solve(0,0)
= max(Fighting, Learning)
= max(2, 5)
= 5
```

7. Where Memoization Helps

Look carefully at the tree:

```
solve(0,1)
```

appears multiple times.

For example:

```
solve(1,0)
```

```
└─ solve(0,1)
```

and:

```
solve(1,2)
└─ solve(0,1)
```

Without memoization, we calculate it again.

With:

```
C++
if (dp[day][last] != -1)
    return dp[day][last];
```

the first time:

```
solve(0,1)
┆
calculate
┆
dp[0][1] = 5
```

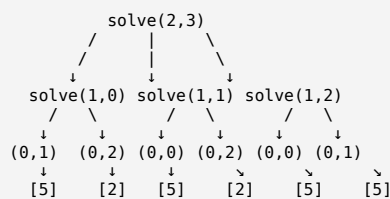
The next time:

```
solve(0,1)
┆
dp[0][1] already exists
┆
return 5 immediately
```

That's the whole point of memoization.

8. Memoization Tree With Cached States

You can visualize it like this:



The repeated states:

```
solve(0,1)
solve(0,2)
solve(0,0)
```

are **not recalculated**.

9. How the Answer Becomes 11

Let's calculate upward.

solve(1,0)

Yesterday = Running.

Choices:

```
Fighting:
1 + solve(0,1)
= 1 + 5
= 6
```

```
Learning:
1 + solve(0,2)
= 1 + 2
= 3
```

Therefore:

```
solve(1,0) = max(6,3) = 6
```

solve(1,1)

Yesterday = Fighting.

```
Running:
3 + solve(0,0)
= 3 + 5
= 8

Learning:
1 + solve(0,2)
= 1 + 2
= 3
```

Therefore:

```
solve(1,1) = 8
```

solve(1,2)

Yesterday = Learning.

```
Running:
3 + solve(0,0)
= 3 + 5
= 8

Fighting:
1 + solve(0,1)
= 1 + 5
= 6
```

Therefore:

```
solve(1,2) = 8
```

10. Finally solve(2,3)

No previous activity, so all three are allowed.

Choose Running

```
3 + solve(1,0)
= 3 + 6
= 9
```

Choose Fighting

```
3 + solve(1,1)
= 3 + 8
= 11
```

Choose Learning

```
3 + solve(1,2)
= 3 + 8
= 11
```

Therefore:

```
solve(2,3)
= max(9, 11, 11)
= 11
```

So the answer is:

```
11
```

One optimal sequence is:

```
Day 0 → Learning = 5
Day 1 → Running = 3
Day 2 → Fighting = 3

Total = 5 + 3 + 3 = 11
```

11. The Most Important Thing to Remember

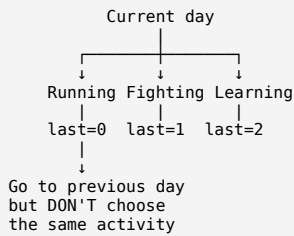
For memoization, remember this state:

```
solve(day, last)
```

Ask yourself:

"I'm currently at this day. What activity did I do last time?"

Then:



The recurrence is:

```
C++
for (int activity = 0; activity < 3; activity++) {
    if (activity != last) {
        int points =
            mat[day][activity] +
            solve(day - 1, activity, mat, dp);
        maxi = max(maxi, points);
    }
}
```

Complexity

There are only:

```
n × 4
```

possible (day, last) states, and each state tries 3 activities.

Therefore:

```
Time = O(n)
Space = O(n)
```

The 4 comes from:

```
0 → Running
1 → Fighting
2 → Learning
3 → No previous activity
```

Key DP pattern: solve(day, last choice) — this pattern is extremely useful for problems where you choose an option at every step but **cannot repeat the previous choice**.

ChatGPT can make mistakes. Check important info.